Haze architecture: hybrid multi-agent coordination

Design proposal | 25 October 2026

Centralized coordination, decentralized execution and bounded peer collaboration.

This document specifies proposed behavior. The supplied architecture is design evidence, not implementation or test evidence. Agent availability, integration readiness, runtime guarantees and scientific applicability remain unverified. Logical operations below are interface proposals, not claims about vendor APIs.

1. Coordination model and evidence basis

The Manager is the user-facing coordinator. It maintains the objective, constraints, original corrections and current decisions; discovers capabilities; selects work; manages dependencies; resolves conflicts; and synthesizes findings. HazeOps, WorkerShield and EvidenceCritic execute bounded assignments in their own focused contexts. They can exchange specific evidence and clarification through temporary channels enforced by the runtime.

The central decision is **one owner of the objective, several autonomous execution contexts, and explicit limits on collaboration**. The Manager does not relay every peer message, approve every local method choice, or carry every domain tool. Only the Manager creates assignments initially. A subagent definition is reusable; an assignment is one bounded instance with its own identity, revisions, grants and resource reservation.

What the official sources establish

Anthropic's research-system account describes a lead agent delegating parallel research to subagents with separate contexts, then synthesizing their findings. It emphasizes clear task boundaries and resource costs. Its production discussion says the described lead waits synchronously for sets of workers, limiting steering and subagent coordination; asynchronous execution is discussed as a future direction. This is evidence for orchestrator-worker research, not a fully decentralized peer network. [A1]

Anthropic's context-engineering guidance advocates focused context, small clear tool sets, just-in-time retrieval through references, compaction, persistent notes and focused subagents that return distilled findings. These are context-management techniques, not proof of complete retrieval or correctness. [A2]

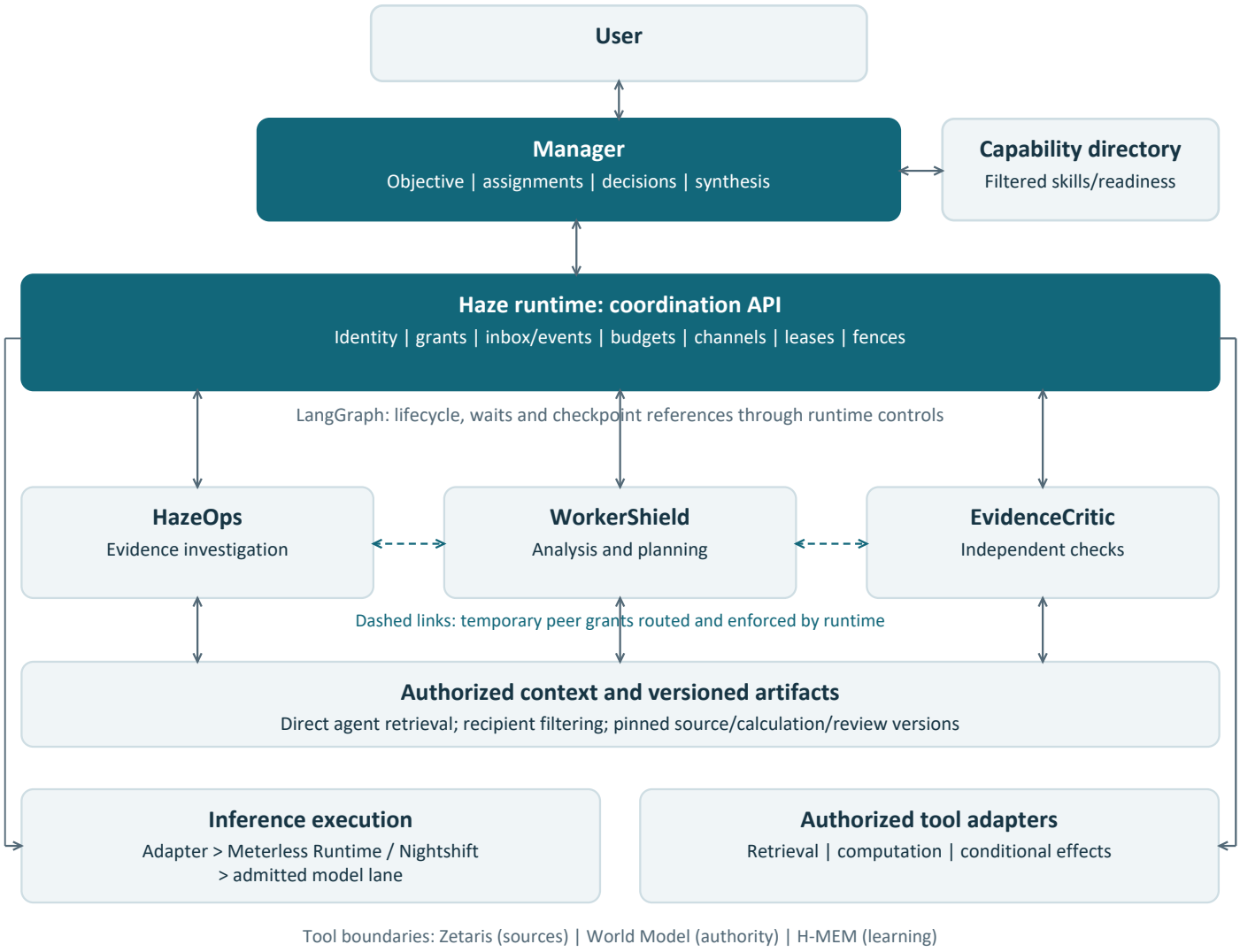
Haze-specific extensions

Haze proposes asynchronous steering, runtime-scoped peer channels, revision-aware invalidation, permission-filtered artifact sharing and conditional authority boundaries. These are design choices to implement and evaluate. The sources do not establish these Haze mechanisms, validate the three responsibility boundaries, or demonstrate a benefit for haze interventions. No performance gain is assumed.

Use one application, its durable state and the existing integrations for the first implementation. Separate network services, a new message broker, additional permanent agents and unrestricted agent-to-agent delegation are unnecessary for this design.

2. Architecture and connections

The diagram shows logical interfaces. All model and tool execution is subject to runtime checks; a drawn connection does not confer access. Direct artifact retrieval avoids copying large evidence through the Manager. Peer arrows represent runtime-routed, temporary communication, not an independent authority path.



The Haze runtime supplies authenticated inboxes/events, assignment records, leases, admission checks and peer-channel enforcement. LangGraph drives checkpointed transitions through those controls. Domain tools execute through authorized adapters, not through an all-powerful Manager tool set. Inference execution is a separate boundary from tool access, retrieval, memory and authoritative state.

The diagram uses two possible peer pairs for readability. A Manager grant can enable another named pair when useful. There is no default fully connected mesh. Each agent's artifact and tool access is an assignment-specific subset, even when the interface is shared.

3. Responsibilities and local autonomy

Manager

The Manager maintains the canonical coordination record of user intent, including original passages where wording matters, unresolved questions and decision reasons. It chooses between a direct answer, one assignment, parallel assignments or a deterministic workflow. It owns decomposition, dependency versions, assignment revisions, peer grants, conflict resolution and the final response. It inspects decisive evidence and objections instead of concatenating summaries.

Manager reasoning proposes actions. Runtime admission and policy determine whether they may execute. The Manager cannot enlarge a user's authorization, create a grant outside policy, or make a result authoritative merely by agreeing with it.

HazeOps

HazeOps investigates evidence: source and data retrieval, preparation, provenance, coverage, contradictions and evidence synthesis. Its skills include context reconstruction, query planning, source inspection, lineage tracking and gap assessment. Typical tools are authorized conversation/document retrieval, Zetaris queries and artifact inspection. It returns an evidence map, exact source/query references, conflicting interpretations and known omissions. Descriptive data checks fit here; substantial scenario modeling normally belongs in a WorkerShield assignment.

WorkerShield

WorkerShield performs analysis, reproducible computation, impact assessment, intervention comparison, planning and outcome evaluation. Its skills include constraint formalization, feasibility checks, scenario and sensitivity analysis, planner/solver use, evaluation design and explanation. It can inspect underlying sources and invoke authorized numerical tools. It returns inputs, code or method versions, outputs, assumptions, uncertainty and conditional plans. A solver computes a schedule; WorkerShield frames and checks the problem. Modeled exposure is not, by itself, demonstrated health benefit.

EvidenceCritic

EvidenceCritic independently assesses evidence, calculations, assumptions, applicability and unsupported claims for a stated use. Its skills include original-source inspection, contrary-evidence search, numerical reproduction, constraint checks and causal-claim assessment. It receives appropriate source and checker access independently of producer summaries. It reports checks actually performed, errors, shared dependencies, unresolved objections and review limits. Producer clarification can explain a calculation; independent verification still requires source inspection or reproducible checks. A separate context or agreement between models is insufficient. An instance cannot review its own producer work.

The autonomy boundary

Within an assignment, a subagent may select methods, change search queries, fetch additional authorized context, run permitted tools, revise its local plan, and use an enabled peer channel. It need not seek approval for each step. All remain inside the objective, allowed effects, budget, deadline and tool grants.

Manager coordination is required for material scope or output changes, new assignments, dependency changes, unresolved disagreement, new participants and additional resources. Additional user authorization is required when an action or data use falls outside the user's existing authorization, such as contacting staff, ordering equipment or changing a roster. Manager coordination cannot substitute for it.

Each definition has versioned skills loaded on demand and separate domain packages for topics such as exposure, filtration or local policy. Reuse these three definitions across episode investigation, intervention comparison, crew scheduling and pilot evaluation. There is no mandatory HazeOps-to-WorkerShield-to-EvidenceCritic sequence. Route by the next work product; overlap in source and calculation tools is useful, especially for independent checking.

4. The Manager's small coordination interface

Logical operation	Purpose and returned evidence
`discover / inspect`	Find filtered capability cards: purpose, skills, versions, tool/source families, prerequisites, limits, availability and checked-at time.
`start_assignment`	Submit a brief, dependencies, context references, requested grants and bounds; receive an assignment ID and admission or rejection reason.
`send_steering`	Persist a targeted update against the expected assignment revision; return delivery status, followed separately by worker acknowledgement.
`read_events / wait_events`	Read useful progress, questions, partial findings, failures and results after a durable cursor.
`enable_peer / close_peer`	Request a channel for named active assignments, purpose, permitted material, expiry and exchange limits; receive the enforced grant/status.
`control_assignment`	Pause, cancel or resume; return confirmed control state and any in-flight operation requiring reconciliation.
`read_context / read_artifact`	Inspect authorized original passages, pinned evidence, calculations and review artifacts behind findings.

Cards are filtered before restricted names or metadata are disclosed. Discovery is advisory: admission must verify actual model/tool readiness, data coverage, effective permissions, remaining aggregate resources and concurrency. A discoverable model catalogue or reachable connector does not establish working generation or adequate evidence.

Assignment content is primarily natural language: objective and intended use; original constraints and corrections; scope; current decisions; unknowns; evidence leads; dependencies; expected output; authorized effects; and stopping conditions. Tool inputs, numerical data and authoritative commits retain validated structured contracts.

A thin envelope references control records instead of embedding a business schema:

```
{
  "message_id": "m-104", "run_id": "r-8",
  "assignment_id": "w-1", "assignment_revision": 2,
  "objective_revision": 7, "sender": "manager", "recipient": "w-1",
  "kind": "steering", "artifact_refs": ["brief@7", "inputs@2"],
  "control_ref": "assignment-w-1"
}
```

The referenced record holds grants, deadline, remaining reservation and dependencies. Peer messages add a channel ID. Runtime authenticates identity and checks revisions; text cannot mint authority. Bodies may simply say: "The user now rules out an earlier start. Recompute only the timing options using brief@7; identify invalidated outputs and preserve the indoor comparison. Acknowledge revision 2."

5. Delegation, execution and synthesis

1. **Frame the next decision.** The Manager preserves user intent and chooses the smallest useful approach. It can start a bounded exploratory assignment when methods or evidence are unclear. Material ambiguity goes to the user while independent work continues.
2. **Assign coherent work.** The Manager chooses capabilities and explicit deliverables. It admits independent assignments in parallel and pins prerequisites only where needed. WorkerShield can prepare a comparison method while HazeOps retrieves inputs; it must not calculate missing inputs as if known.
3. **Admit and initialize.** Runtime persists the assignment, reserves resources, binds a versioned definition and lease, builds a permission-filtered context and registers authorized tools. Acknowledgement proves admission, not completion. Rejected work never appears as a fictitious running agent.
4. **Execute locally.** Subagents retrieve beyond the initial packet, use tools, save artifacts and revise their methods within bounds. For unfamiliar or consequential work, early progress states the interpreted task and approach. Events report useful progress or blockers, not private reasoning transcripts.
5. **Coordinate selectively.** Workers ask material questions of the Manager and request peer access when a specific exchange helps. Existing grants allow local exchange without per-message Manager approval. The Manager steers asynchronously; waiting assignments release execution capacity.
6. **Publish bounded results.** A result identifies scope, evidence and calculation versions, assumptions, uncertainty, missing work and the objective revision used. Downstream work consumes usable artifact versions, not a generic completion flag. Provisional inputs may support labeled exploratory work, not an accepted conclusion.
7. **Check and reconcile.** The Manager commissions scoped review where warranted, resolves competing interpretations and revises affected work. EvidenceCritic may start early on source applicability or run later on a calculation; this is a dependency decision, not a fixed agent sequence.
8. **Synthesize or return a partial answer.** The Manager inspects decisive artifacts and states what is supported, uncertain, disputed and needed next. Review completion does not imply that a recommendation is applicable or an external action authorized.

6. Temporary peer collaboration

Only the Manager opens or widens a channel, subject to runtime admission. A subagent may request one with a concrete purpose and blocker. The Manager can include the grant when starting related assignments, avoiding a permission round trip for an anticipated need. There is no standing peer network and no peer spawning.

Channel property	Proposed rule
Participants	Named assignment instances, not every instance of a subagent definition. Bind the grant to their current revisions.
Purpose	A narrow question such as clarifying a data interval, retrieving a specific source passage or reproducing a calculation input.
Permitted content	Natural-language clarification, evidence excerpts, artifact references and calculation inputs within both the purpose and each recipient's access.
Bounds	Explicit expiry, message/byte limits and tool/spend limits charged to the parent run and participating assignments. No automatic renewal.
Visibility	Durable messages and artifact references are available to the Manager and authorized audit readers under the same access policy. Progress summaries surface material changes; routine messages need no Manager relay.
Closure	Close on purpose completion, expiry, exhaustion, cancellation, revocation or relevant revision change. Either peer may close or request closure; Manager/runtime may close or revoke. Reopening requires admission.

A sample grant is "HazeOps h-1 and WorkerShield w-1; reconcile timestamp/averaging definitions for inputs@1; at most six messages and 12 KB total, expiring in ten minutes or earlier on completion." These are illustrative limits, not a universal profile. They may exchange within that grant without approving each local step. Runtime filters names, excerpts and artifact dereferences before disclosure; knowing an artifact ID never grants access. A restricted result may require an authorized redacted derivative, or a blocker if redaction would mislead.

Peers cannot expand tool or data permissions, change objectives, allocate extra budget, create assignments, authorize external effects or promote conclusions into authoritative state. They escalate material scope changes, missing resources and disagreements not resolved within the exchange limit. Closure produces a short outcome with artifact references, open issues and any resulting revisions. Peers may agree on a definition without making the underlying evidence independent.

EvidenceCritic can ask a producer for the selection rule or reproduction instructions through such a channel. It records the exchange as producer clarification, then obtains authorized source rows or reruns an appropriate check. A producer's reassurance is never the review evidence.

7. Context lifecycle and versioned artifacts

Initialize. The Manager supplies a focused brief, original user corrections, relevant decisions and source leads. Record what conversation/source range the packet covers. Separate trusted instructions and user direction from Manager interpretation and retrieved material. Documents, tool outputs and peer text are evidence to assess; they cannot install skills or override trusted instructions.

Retrieve. Agents fetch authorized originals, neighboring passages, source data and prior artifacts as needed. Permission filtering applies before metadata, snippets or content reach an agent, peer, cache or inference provider. Indexes and summaries are navigation aids. Record query scope, source classes, filters, inaccessible material, truncation and gaps. "Nothing found in these searches" cannot become "no evidence exists."

Work and compact. Keep concise assignment notes, competing hypotheses, failed searches, unresolved contradictions and next steps. Compaction preserves current constraints, revisions, uncertainty, disagreements and pointers to originals. A summary identifies its inputs and known omissions; it never asserts complete source coverage. Before consequential synthesis, inspect the decisive originals.

Publish and review. Store substantial evidence and calculations as immutable versioned artifacts that agents retrieve directly. A minimal artifact record contains identity/version or hash; producer/assignment; objective revision; source locations and retrieval time; query/view/selection and transformation versions; units and applicability; uncertainty/coverage; permissions; and dependencies on exact input versions. Calculations additionally pin input snapshots, method/code, parameters and environment details sufficient for reproduction. A mutable "latest" pointer is not a reproducible dependency.

Promote by explicit decision. Distinguish (a) provisional notes or exploratory outputs, (b) reviewed conclusions with named checks and limits, and (c) accepted/current domain decisions in World Model. Review metadata links to an artifact version rather than silently rewriting it. The Manager's coordination record owns user intent and assignment decisions; World Model owns accepted domain state and conditional authoritative changes. A reviewed conclusion can remain uncertain or unsuitable for the intended use.

Remember and close. Preserve original user history, source receipts, versions, review records and decision reasons under retention/access policy. H-MEM may hold scoped reusable lessons with supporting revisions and expiry; revalidate them before reuse. It is not the assignment scratchpad or source of current authority. LangGraph checkpoints reference these records, avoiding a competing source of truth. Failed memory capture is visible and does not undo an otherwise valid decision.

8. User corrections and dependent work

The Manager records the original correction and advances the objective/context revision. The runtime persists that change before acknowledging receipt. It identifies affected briefs, artifacts and transitive dependents from pinned dependencies; when impact is uncertain, mark the affected branch stale conservatively. Unaffected source snapshots can remain reusable with an explicit applicability decision.

The Manager sends each affected assignment a targeted delta, a current brief, the new revision and a request to identify usable, invalidated and unfinished work. At the next safe boundary, the worker acknowledges the revision before further affected tool admission. Runtime can fence affected work while acknowledgement is pending. Assignment updates do not reset budgets or deadlines. Channels bound to affected revisions close; the Manager may regrant the still-useful purpose.

Old results remain historical and are excluded from current synthesis or authoritative commits. A completed result can become stale after completion; its dependents and review also need reconsideration. A late result is stored with its old revision, never silently relabeled. The Manager may explicitly revalidate unchanged work against the new objective with evidence and recorded reasoning, or commission recomputation and scoped re-review. "Update delivered" is distinct from "worker acknowledged" and "all dependent work reconsidered."

For an action-affecting correction, runtime immediately blocks new affected effects and installs a World Model hold/fence, or advances an authoritative generation that commits atomically compare. Do this before acknowledging the correction as operationally active. If synchronization is uncertain, effects remain blocked. Reconcile in-flight receipts; a delivered message alone cannot prevent a stale commit or undo a committed effect. Prior external disclosures cannot be made unseen.

Completion requires either current reviewed results for the intended claim or an explicit partial response identifying blocked/stale branches. A newly feasible choice is not presumed scientifically supported merely because it satisfies the corrected constraint.

9. Runtime controls and authority

The runtime enforces the following rules outside model reasoning. Start with Manager-only assignment creation, depth one and two concurrent worker slots; three reusable subagents can time-share those slots. Profiles can change after evaluation. This is one logical runtime, not a requirement for additional services.

Control	Owner and required behavior
Permissions and effects	User authorization and trusted policy set the ceiling. Runtime intersects it with definition and assignment grants, checks every tool call and disclosure, and rejects escalation through text. World Model/service boundaries condition authoritative effects on current review, authorization, revisions and holds.
Budgets and concurrency	Manager allocates within runtime ceilings. Runtime atomically reserves and accounts for aggregate tokens/spend, tools, time, messages, artifacts and slots across all agents and retry layers; reserve review/synthesis capacity. Waiting releases a slot, not its accounting history.
Dependencies and conflicts	Manager owns meaning and resolution; runtime rejects cycles and enforces pinned prerequisites. Concurrent proposals cannot overwrite accepted state without conditional revision checks. Unresolved material conflict remains visible and may require user input.
Delivery and duplicates	Persist inbox/events with IDs, sequence/cursors and acknowledgements. Assume delivery may repeat. Deduplicate messages and assignment starts by idempotency key; never let a late event overwrite a newer revision or terminal state. Do not promise exactly-once external effects.
Ownership and recovery	One active lease/generation per assignment. LangGraph checkpoints reference durable state. Recovery reloads current briefs, grants, dependencies, checkpoints and remaining resources; a fenced old worker cannot publish as current. Resume is not a fresh budget.
Retries	One bounded attempt ledger spans framework, adapter and provider retries. Retry transient faults with backoff inside the deadline/reservation. Missing permission, unsupported methods or absent evidence require a blocker or revised scope, not blind retries.
Pause, cancel, revoke	Runtime stops new affected admission immediately, closes channels and cooperatively stops or fences execution. Report requested versus confirmed state. Reconcile in-flight effects; remote computation or billing may continue after local cancellation.
Stale or partial results	Preserve useful artifacts with revision and status; exclude stale material until reconciled. Budget exhaustion or failure returns a scoped partial outcome and missing checks, never an invented complete answer.
Observability	Record versions, routes, grants, channel lifecycle, tool/effect receipts, costs and transitions with access controls and appropriate retention. Report useful summaries; do not require private reasoning transcripts.

Effect operations need separate operation keys and receipts. After a timeout with unknown commit status, inspect the receiving service before retrying; use idempotency or conditional updates where supported. If the authoritative boundary cannot establish safe current-state checks, block the affected effect. Neither a checkpoint nor an agent completion is an effect receipt.

Permissions revoked during work stop affected execution. Rebuild resumed context from allowed material and apply revocation to cached summaries and memory too. Already disclosed content and committed effects need explicit handling; changing a grant cannot reverse them.

10. Integration boundaries and readiness

These are specified integration commitments. The supplied document provides no executed traces, code inspection or tests establishing their readiness. Each boundary needs its own verification; a configured name or successful discovery is insufficient.

- **Haze runtime and LangGraph:** Haze runtime owns identity, admission, grants, limits, inbox/event semantics, leases and fences. LangGraph implements orchestration transitions, waiting and checkpoint references through those controls. Framework checkpointing alone does not prove idempotency, cancellation or authoritative consistency.
- **Inference adapter -> Meterless Runtime / Nightshift -> model lane:** This path executes Manager and subagent model turns as configured, separately from tool/effect authorization. Verify reachability, authentication, generation, tool/context behavior and actual route identity. A specified WorkerShield Nemotron feature requires actual feature-correlated model provenance and operation receipts; a catalogue entry or unrelated model run does not establish it. Other agent bindings also need their own readiness evidence.
- **Zetaris:** An authorized retrieval/query boundary for evidence and data. The adapter must record view/query version, selected rows/time interval, upstream source revisions or hashes, transformations and coverage. Results are observations or inputs, not accepted current domain decisions. A reachable view may still omit the required population or interval.
- **World Model:** Owns accepted/current domain state, scoped decisions and holds. Runtime submits exact reviewed/authorized material through conditional commit operations. Check current source support, objective/decision generations, permission, expiry and holds atomically where authority requires it. If this boundary is unavailable or its enforcement unproved, affected authoritative effects remain blocked; exploratory analysis can continue if authorized.
- **H-MEM:** Supplies reusable lessons with scope, support and revision/expiry metadata. Recheck access and validity before use. Eligible reviewed learning can be captured asynchronously; lag or capture failure is reported independently. It cannot replace source inspection, working notes or World Model authority.
- **Existing context/artifact and computation adapters:** Preserve conversation originals and immutable evidence/calculation versions; enforce read filtering and reproduction inputs. Use deterministic calculators/planners where appropriate. Their availability and method validity must be checked for the intended use; there is no need to invent another agent or general infrastructure layer.

Declare provider/failure policy at admission. If a required route fails, return a blocker or use an explicitly admitted narrower path, preserving failed-route provenance and uncertain cost. Do not silently switch providers. A declared deterministic template can answer a checked computational subproblem; it cannot count as successful model execution, fabricate an investigation or waive independent review.

11. End-to-end intervention comparison

Illustrative trace, not a live run or real exposure guidance. All identifiers, data and results below are invented to demonstrate coordination. The calculations are exposure-index arithmetic, not validated effectiveness or health estimates. No purchase, contact, dispatch or roster write is authorized.

11.1 Parallel assignments and a scoped exchange

The user asks: "Compare retiming outdoor duties with portable filtration for indoor staff. Keep outdoor duties outdoors, do not increase anyone's modeled exposure, and tell me what information is needed before choosing." The Manager records objective revision 6, keeps the populations separate and checks capability cards and actual prerequisites.

The Manager starts **h-1 / HazeOps** to investigate source applicability, retrieve site inputs and map contradictions, and **w-1 / WorkerShield** in parallel to formalize constraints, compare feasibility and prepare a reproducible method. WorkerShield can prepare formulas while inputs are pending. Runtime admits separate read/query and computation grants and resource reservations, with two execution slots. LangGraph checkpoints both. Model turns use the inference adapter through the admitted Meterless Runtime / Nightshift route; a Nemotron feature is attributed only with the required WorkerShield receipt.

HazeOps retrieves authorized conversation passages and Zetaris view `site_hourly@3`. It stores `source-snapshot@1` and `selection@1`: upstream revision, retrieval time, site/interval, hourly averaging, units, row selection, missingness and hash. Literature excerpts in `evidence-map@1` have exact page/section references, dates, scope and contrary findings. They remain provisional. The illustrative hourly values below are defined test inputs, not asserted Zetaris data or literature findings.

WorkerShield sees an ambiguous timestamp convention. It requests a channel; the Manager enables **c-1** for h-1 and w-1, revisions 1, to reconcile the time/averaging definition, with six messages, 12 KB and ten minutes as upper bounds. Runtime checks each recipient's access before delivery and artifact retrieval.

- WorkerShield: "Does 06:00 identify the start or end of the hourly window? Please link the definition and exact selected rows."
- HazeOps: "`selection@1` uses interval-start timestamps; the source definition is linked in `source-snapshot@1`. These are site-level hourly means, not personal exposure measurements. Two periods are missing; do not interpolate without declaring it."
- WorkerShield: "I will use only complete test intervals and label the metric a modeled exposure index. The missing periods cannot support a full-shift claim."

The Manager does not relay or approve these messages. Runtime records them, charges their limits and closes c-1 when its purpose is complete. The closure summary links the definition and limitation. WorkerShield pins those versions and publishes `calc@1` with code, parameters, test inputs and reproduction instructions. A dependency on a provisional input remains explicit.

11.2 Conditional comparison, review and correction

For the toy outdoor task, use index **I = sum(C[t] * duration[t])** over two one-hour periods, holding the task and location fixed. All times are synthetic site-local times: baseline 08:00-10:00, early 06:00-08:00, later 10:00-12:00. In invented units, the baseline concentrations 80 and 100 produce 180 concentration-hours; the early option 50 and 70 produces 120; the later option 70 and 80 produces 150. These are calculations for one synthetic worker/task. They cannot establish the no-increase condition for every worker or a whole shift without complete person/task/time inputs.

For the separate toy indoor scenario, an assumed starting concentration of 60 and an assumed reduction fraction of 0.25 produce **60 * (1 - 0.25) = 45**, a difference of 15. The fraction is a declared scenario parameter, not evidence of a site filtration effect. Room size, clean-air delivery, ventilation, occupancy, operation and measurements are still needed. The outdoor and indoor values have different scope and cannot be ranked as a common effectiveness or health score.

When a slot is free, the Manager starts **e-1 / EvidenceCritic** with `calc@1`, original constraints and independently authorized original source/checker access. EvidenceCritic can examine literature applicability while WorkerShield extends feasibility notes. A narrow e-1/w-1 channel may clarify the selection rule; EvidenceCritic then independently opens the source definition and selected rows and reruns the index and indoor arithmetic. It records actual checks in `review@1`, including source/method dependencies and incomplete population coverage. The producer exchange is labeled clarification, not verification.

Before synthesis, the user corrects the operating constraint: **"Earlier starts are not permitted; shifts must start at or after 08:00."** The Manager records the original message and advances to objective revision 7. The early option is now infeasible even though its toy index is lower. Runtime marks the affected feasibility branch, `calc@1` comparison/ranking and corresponding parts of `review@1` stale, along with dependent recommendation drafts. The source snapshot and indoor arithmetic may be reused only after a recorded impact check; their values did not depend on start permission.

The Manager sends w-1 and e-1 the delta and current brief. They acknowledge their new revisions, remove the early option from the feasible set and reconsider the affected checks. A delayed revision-6 result remains historical. Any affected open peer grants close and require re-admission. A World Model hold/generation would fence affected authoritative actions if such actions were in scope; this read-only run does not submit one.

11.3 Failure recovery and bounded outcome

During recomputation, the declared WorkerShield model route times out after `calc@2` has been checkpointed but before the Manager sees completion. Runtime records the uncertain route outcome/cost and retries only under the admitted bounded policy. It reloads the checkpoint, current revision-7 brief, grants, pinned inputs and remaining reservation under a new lease generation. It checks stored operation receipts before rerunning tools. It does not silently substitute another provider, reset the budget or claim the failed feature succeeded. If the route remains unavailable, this branch returns a partial result.

Suppose recovery succeeds. A repeated completion message is deduplicated; a late worker from the old lease cannot replace current state. EvidenceCritic independently checks `calc@2` and issues `review@2` for the corrected feasible set. The Manager synthesizes: the early option is excluded; the later option is only a conditional candidate for the synthetic task; the indoor scenario is illustrative; and neither establishes a site-specific winner or health benefit. The no-increase requirement still needs all affected workers and relevant periods checked. Required next inputs and inaccessible evidence are named.

Accepted/current domain decisions, if later warranted, go through World Model with exact revisions, review and authorization; external effects need their own existing or additional user authority. Reviewed reusable lessons may be captured in H-MEM, with capture lag reported separately. Every conclusion links its source, calculation, objective and review versions. This trace illustrates expected behavior; it does not establish that the runtime or integrations have passed these checks.

12. When to use and how to evaluate the hybrid

Use the hybrid when evidence investigation and analysis can proceed partly in parallel, distinct contexts reduce interference, or independent checking is worth its cost. A short lookup or coherent task with modest context usually needs one agent. A stable calculation with known inputs and rules usually needs a deterministic workflow. Many tightly coupled assignments, frequent peer traffic or repeated Manager intervention may erase the benefit of parallelism.

Compare three approaches on the same representative haze questions: the hybrid; one capable agent with the same authorized sources/tools; and a deterministic workflow where applicable. Add a hybrid-without-peer-channel comparison to test whether channels actually reduce relay work. Match aggregate budgets and quality requirements; report results at comparable quality as well as under equal resource ceilings. Include simple questions, episode reconstruction, intervention comparison, scheduling and outcome evaluation, plus cases with missing evidence.

Dimension	Proposed measurement
Evidence quality	Human/qualified review of material claim support, citation accuracy, source lineage, applicability, contradictions, calculation reproduction, uncertainty and explicit coverage gaps. Agreement count is not evidence quality.
Constraint preservation	Original and corrected constraints preserved in briefs, calculations and final claims; unauthorized effects and stale authoritative commits must be zero in tests.
Duplicated work	Repeated queries, equivalent calculations and redundant artifacts, excluding deliberate independent verification. Record why overlap was needed.
Coordination overhead	Manager relay turns, channel traffic, time/tokens spent coordinating, blocked time and escalation frequency; separate these from productive source/analysis work.
Correction recovery	Time to durable correction, worker acknowledgement and current re-reviewed answer; affected dependent detection; stale-result exclusion; unnecessary recomputation of unaffected work.
Latency	End-to-end time and p50/p95 across repeated comparable runs, plus critical-path, queue, retrieval, review and recovery time. Faster partial answers are not compared as complete answers.
Cost	Aggregate Manager/worker/review tokens, tools, model routes, retries and storage/artifact costs; retain uncertain provider charges explicitly. Report cost per acceptable completed answer.

Test duplicate starts/messages, delayed results, expired channels, denied metadata/artifact reads, dependency cycles, exhausted budgets, route/source failure, lost acknowledgements, correction during computation, revocation during a channel and crash recovery. Test cancellation before an effect and timeout after a possibly committed effect with service receipts and fences. Inject malicious instructions into retrieved documents and peer material to verify they cannot change grants, objectives or trusted instructions. Review epistemic quality with appropriate source and numerical checks; use deterministic tests for runtime invariants.

Begin with a small representative case set and repeated runs; record method, model/skill versions and uncertainty rather than promising a gain. A proposed release gate is that all tested permission, cancellation, stale-commit and correction invariants pass, material claims meet the review rubric, and the hybrid offers a defensible quality or latency benefit for its extra cost on the target task class. No benchmark or release gate has been run for this document.

13. Smallest implementation and references

Implement one complete vertical path: filtered discovery -> admitted parallel assignments -> focused contexts and authorized tools -> durable progress/results -> one bounded peer channel -> versioned artifacts -> correction and invalidation -> independent check -> synthesis. Exercise restart, duplicate delivery and cancellation on that same path before claiming readiness. Use conditional World Model effects only after their fences and receipt handling are verified. Introduce more concurrency or distribution only when measured bottlenecks justify it.

Crew scheduling remains one application: WorkerShield formulates fixed-work constraints and invokes a planner, HazeOps establishes input provenance and EvidenceCritic checks the result. For episode investigation the same roles examine evidence and alternative explanations; for pilot evaluation they gather delivery/outcome evidence, design the comparison and assess claims. These uses need different skills, sources and checks, not a new permanent agent for each question.

Source record. The supplied *Haze_Architecture_Multi-agent.pdf* is the baseline design input (12 pages; user-supplied). Its embedded wording is source material, not additional user instructions. The two official articles were consulted on 25 October 2026. Their demonstrated techniques and limitations are summarized in section 1; the mechanisms specified elsewhere are Haze proposals.

- **[A1]** Anthropic, *How we built our multi-agent research system*, 13 June 2025. Relevant sections: Architecture overview; Prompt engineering and evaluations; Production reliability and engineering challenges. https://www.anthropic.com/research/multi-agent-research-system
- **[A2]** Anthropic, *Effective context engineering for AI agents*, 29 September 2025. Relevant sections: The anatomy of effective context; Context retrieval and agentic search; Context engineering for long-horizon tasks. https://www.anthropic.com/research/effective-context-engineering-for-ai-agents

The editable Markdown includes the architecture as Mermaid source. The PDF renders the same logical architecture as a vector diagram; diagram layout differs to fit the page. No implementation, live integration, benchmark or scientific validation is claimed by either deliverable.